Chapter 6

Synchronization Tools

SEMAPHORE

Md. Mahmudur Rahman
Assistant Professor
Institute of Information Technology

Semaphore

- Semaphore was proposed by Dijkstra in 1965, which is a very significant technique to manage concurrent processes by using a simple integer value, which is known as a semaphore.
- A semaphore is simply an integer variable that is shared between threads.
- This variable is used to solve the critical section problem and to achieve process synchronization in the multiprocessing environment.
- Semaphores are of two types:
- **Binary Semaphore –** This is also known as a **mutex lock**. It can have only two values – 0 and 1. Its value is initialized to 1. It is used to implement solutions to critical section problems involving multiple processes.
- **Counting Semaphore –** Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Binary Semaphore

- First, look at two operations that can be used to access and change the value of the semaphore variable.
- Some points regarding P and V operation:
- P operation is also called **wait** and V operation is also called **signal**.
- Both operations are atomic and semaphore(s) is always initialized to one. Here atomic means that the variable on which read, modify and update happens at the same time/moment with no pre-emption i.e. in-between read, modify and update no other operation is performed that may change the variable.
- A critical section is surrounded by both operations to implement process synchronization. See the below image. The critical section of Process P is in between P and V operations.

```
P(Semaphore s){
    while(S == 0);   /* wait until s=0 */
  s=s-1;
}

V(Semaphore s){
        s=s+1;
}
```

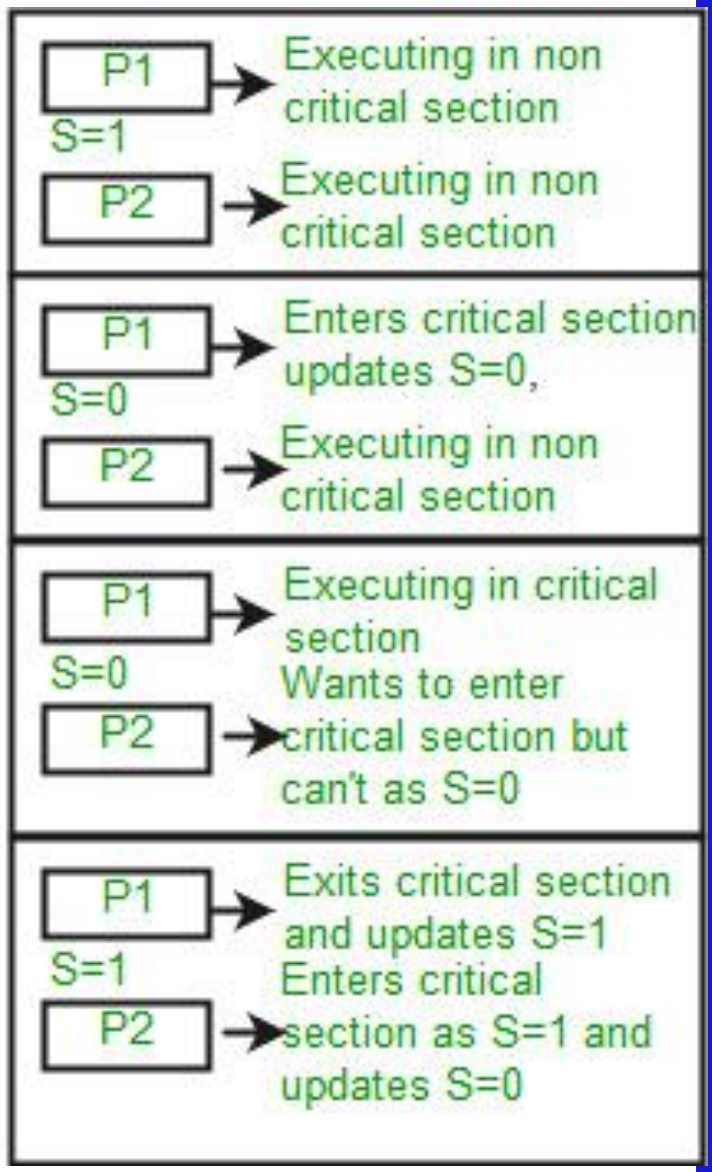
Note that there is Semicolon after while. The code gets stuck Here while s is 0.

Process P

```
// Some code
P(s);
  // critical section
V(s);
  // remainder section
```

Binary Semaphore

- Now, let us see how it implements mutual exclusion.
- Let there be two processes P1 and P2 and a semaphore s is initialized as 1.
- Now if suppose P1 enters its critical section then the value of semaphore s becomes 0.
- Now if P2 wants to enter its critical section then it will wait until s > 0, this can only happen when P1 finishes its critical section and calls V operation on semaphore s.
- This way mutual exclusion is achieved. Look at the below image for details which is a Binary semaphore.



Counting Semaphore

- A counting semaphore is a synchronization mechanism that allows multiple processes to access multiple instances of a shared resource.
- It maintains a count that represents the **number of available instances of the resource**. When a process wants to access the resource, it performs a "wait" operation, which decrements the count. And when a process releases the resource, it performs a "signal" operation, which increments the count.

- Now suppose there is a resource whose number of instances is 4. Now we initialize S = 4 and the rest is the same as for binary semaphore.
- Whenever the process wants that resource, it calls P or waits for function and when it is done it calls V or signal function.
- If the value of S becomes zero, then a process has to wait until S becomes positive.
- For example, Suppose there are 4 processes P1, P2, P3, P4, and they all call wait operation on S(initialized with 4).
- If another process P5 wants the resource then it should wait until one of the four processes calls the signal function and the value of the semaphore becomes positive.

Producer-Consumer Problem

- In the producer-consumer problem, let us say there are two processes, one process writes something while the other process reads that.
- The process which is writing something is called producer while the process which is reading is called a consumer.
- To read and write, both of them are using a buffer.

- The **producer** produces the item and inserts it into the buffer.
- The value of the global variable count increased at each insertion. If the buffer is filled and no slot is available, then the producer will sleep; otherwise, it keeps inserting.
- On the **consumer's end**, the value of the count decreased by 1 at each consumption. If the buffer is empty at any point, the consumer will sleep; otherwise, it keeps consuming items and decrements the count by 1.

Producer-Consumer Problem

- Here, both the producer and the consumers share the same resource (buffer). So, it is a critical section problem & we need process synchronization.
- Producer and consumer cannot accept the buffer simultaneously.
- To solve the producer-consumer problem we have to maintain the following-

Producer	**Consumer**
1. Check overflow 2. Mutual exclusion 3. Buffer Update	1. Check underflow 2. Mutual exclusion 3. Buffer Update

wait(s)
{
while(s<=0);
s--;
}
If resource unavailable (`s <= 0`)
→ keep waiting
	signal(s)
	{
	s++;
	}

Producer-Consumer Problem Using Semaphore

- Let consider, empty E=n, semaphore S=1, Item I=0 [n=buffer size]

Producer	Consumer

do{
wait (E); //overflow checking
wait (S); //Entry section
add (); //CS
signal(S); // leave CS
signal(I); // increase filled items
} while (true);
	do{
	wait (I); //underflow checking
	wait (S); //Entry section
	consume (); //CS
	signal (S); //Exit CS
	signal (E); // increase empty slot
	} while (true);

- Initially the producer checked for empty slots and found four, so no overflow occurred. Then the value of the semaphore is changed to 0.
- The producer can access the critical section, change the semaphore to 1, and increment the number of item. if the value of empty becomes zero then it is called an overflow and it is got trapped.
- On the consumer end, initially check for underflow, consume items, and finally change the value of the semaphore and empty.

Producer Consumer Problem Using Semaphore

- Here in this producer-consumer problem, semaphore ensures mutual exclusion and progress.
- No consumer can enter the critical section if a producer is there. The same is for any producer.
- Again, after using the critical section, a producer or consumer updates the semaphore, which ensures that no one is preventing another.

Reader Writer Problem Using Semaphore

- Suppose, we have a notice board where a teacher can put a notice and students can read the notice.
- When the teacher updates the notice board no student can read the notice until the teacher has completed it.
- Here, the teacher is the writer and the students are the reader.

Writer	**Reader**
1. One writer executes in the critical section. 2. Readers will not execute in the critical section.	1. Multiple readers can execute in the critical section. 2. Writer will not execute in the critical section.

- This is the same as the read and write problems in the operating system, where one program writes and the other reads, and read and write don't interfere with each other.

Reader Writer Problem Using Semaphore

- Let consider, write=1, read=1, read_count=0.

Writer	Reader

wait (write); //Entry section
writeFunction (); //CS
signal (write); //Exit section
	wait (read); //Entry section
	read_count++
	if (read_count==1)
	{
	wait (write); //no writer can enter
	}
	signal (read); //multiple reader can access
	
	readFunction () // CS
	
	wait (read); //Exit section
	read_count--
	if (read_count==0)
	{
	signal (write); //now writer can enter
	}
	signal (read);

Reader Writer Problem Using Semaphore

1. If there is a reader inside the critical section, can any other reader access the critical section?
2. If there is a reader inside the critical section, can any writer access the critical section?
3. If there is a writer inside the critical section, can any reader access the critical section?
4. If there is a writer inside the critical section, can any other writer access the critical section?

Practice Problem

- Think about the scenario of an ATM booth, where multiple users can withdraw money at the same time using an ATM. But, only one person can load money inside the ATM, and no user can withdraw money at that time.
- **Design a solution to the problem using Semaphores.**

THANK YOU!